Smart Chilli Crop Intelligence System Using an Autonomous Ground Rover

Project ID: R26-IT-075

Final Group Report

IT22112682 Arachci B N S

IT22108722 KULASEKARA D K M

IT22276032 S G M B Subasinghe

IT22192950 S.N.R Jayawardhana

B.Sc. (Hons) Degree in Information Technology Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

October 2026

Smart Chilli Crop Intelligence System Using an Autonomous Ground Rover

Project ID: R26-IT-075

Final Group Report

IT22112682 Arachci B N S

IT22108722 KULASEKARA D K M

IT22276032 S G M B Subasinghe

IT22192950 S.N.R Jayawardhana

B.Sc. (Hons) Degree in Information Technology Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

October 2026

DECLARATION

I declare that this is my own work, and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Arachci B N S	IT22112682	
KULASEKARA D K M	IT22108722	
S G M B Subasinghe	IT22276032	
S.N.R Jayawardhana	IT22192950	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Name of the Supervisor: Prof. Samantha Rajapaksha

Signature of the Supervisor: ……. Date: ……

Name of the Co-Supervisor: Miss. Chathurya Kumarapperuma

Signature of the Co-Supervisor: …… Date: ……

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, Prof. Samantha Rajapaksha, and co-supervisor, Miss. Chathurya Kumarapperuma, for their guidance and constructive feedback throughout this research project. Their advice helped us refine the research direction, examine the individual components, and develop a coherent approach to the integrated system.

We extend our appreciation to the Sri Lanka Institute of Information Technology and the Department of Information Technology for the opportunity to undertake this research. We also acknowledge the researchers, dataset contributors, and open-source software communities whose resources supported the project.

The cooperation and commitment of all four group members were central to bringing together intelligent navigation and irrigation, crop monitoring, field zone planning, and soil intelligence. Finally, we thank our families and friends for their patience, understanding, and continued encouragement throughout the project.

ABSTRACT

The Smart Chilli Crop Intelligence System investigates an autonomous ground rover as a shared platform for navigation and irrigation, crop disease and condition monitoring, field zone planning, and soil assessment. This group report integrates four component designs with the supplied offline experiments and dashboard prototype. The recorded anomaly classifier obtains approximately 96% accuracy but misses all nine pseudo-labelled test anomalies. The eight-category crop classifier obtains 13.27% test accuracy and requires corrective development. Multispectral K-Means selects two clusters with a final silhouette score of 0.5423, demonstrating feature-space grouping rather than verified chilli field zones. The soil LSTM–XGBoost classifier reports 99.46% agreement with constructed health labels, but its one-timestep inputs do not establish future forecasting. The supplied Flask and MongoDB application organises seeded demonstration records; live inference, cloud deployment and actuator integration are not demonstrated by the available evidence. The report documents datasets, preprocessing, modelling, source figures, interface requirements and proposed testing. It concludes that the project has a coherent architecture and reproducible development artefacts, while independent local validation and measured field operation remain necessary. Priorities are corrected evaluation partitions, calibrated sensors, expert-reviewed labels, coordinate-preserving data fusion and supervised navigation, irrigation, seeding and treatment trials.

Keywords: ***Chilli cultivation; Autonomous ground rover; Irrigation; Crop monitoring; Management zones; Soil intelligence; Machine learning; Integration.***

LIST OF FIGURES

Figure 2.1: Intended integrated system architecture 6

Figure 2.2: Sample chilli category images 11

Figure 2.3: Proposed observation and action cycle 16

Figure 3.1: Component 1 test confusion matrix 20

Figure 3.2: Crop classifier training and validation curves 21

Figure 3.3: Crop classifier test confusion matrices 21

Figure 3.4: Candidate cluster number evaluation 22

Figure 3.5: Spectral clusters in principal component space 22

Figure 3.6: Soil classifier LSTM learning curves 23

Figure 3.7: Soil classifier test confusion matrices 24

LIST OF TABLES

Table 1.1: Component allocation 3

Table 1.2: Specific research objectives 3

Table 2.1: Architecture layers and implementation status 6

Table 2.2: Component 1 dataset and model specification 8

Table 2.3: Component 2 image split counts 10

Table 2.4: Component 3 clustering specification 13

Table 2.5: Component 4 dataset and hybrid specification 15

Table 2.6: Testing and implementation plan 18

Table 3.1: Recorded component results 19

Table 3.2: Dashboard code inspection findings 25

Table 3.3: Integrated evidence and pending verification 26

Table 3.4: Objective coverage and remaining work 27

Table 3.5: Student component responsibilities and supplied artefacts 28

Table 4.1: Prioritised future work 30

LIST OF ABBREVIATIONS

AI — Artificial intelligence

API — Application programming interface

CNN — Convolutional neural network

EC — Electrical conductivity

EVI — Enhanced vegetation index

F1 — Harmonic mean of precision and recall

GIS — Geographic information system

GNSS — Global navigation satellite system

HDOP — Horizontal dilution of precision

IMU — Inertial measurement unit

IoT — Internet of Things

LSTM — Long short-term memory

MAE — Mean absolute error

ML — Machine learning

NDVI — Normalised difference vegetation index

NDWI — Normalised difference water index

NIR — Near-infrared

NPK — Nitrogen, phosphorus and potassium

PCA — Principal component analysis

RF — Random Forest

RMSE — Root mean squared error

SAVI — Soil-adjusted vegetation index

SWIR — Short-wave infrared

XGBoost — Extreme gradient boosting

1. INTRODUCTION

1.1 Background

The Smart Chilli Crop Intelligence System investigates an autonomous ground rover as a shared platform for crop observation, soil assessment and location-aware farm management. The project is organised into navigation and irrigation, crop monitoring, field zone planning, and soil intelligence components. The individual proposals define complementary responsibilities within project R26-IT-075 at the Sri Lanka Institute of Information Technology [1], [2], [3], [4]. Their common purpose is to organise field evidence into decisions that can be reviewed and traced to a specific place and time.

Digital agriculture research demonstrates the range of machine learning applications in crop, water and soil management [5], [6]. The project's contribution is consequently assessed through the quality of its particular chilli monitoring and integration workflow, rather than through a claim that agricultural AI is new. The supplied sources include four offline experiments, a shared design document and a Flask dashboard prototype [7], [8], [9], [10], [11], [12]. They establish a development foundation but do not collectively prove complete autonomous field operation.

1.2 Background Literature Survey

The literature connects agricultural sensing with both predictive models and operational decision support. Liakos et al. and Benos et al. review applications across crop, soil and water management, showing the breadth of tasks and data sources involved [5], [6]. Kamilaris and Prenafeta-Boldú survey agricultural deep learning and emphasise the relationships among agricultural problems, input data, preprocessing and evaluation [13]. Chlingaryan et al. discuss crop yield prediction and nitrogen estimation, which are relevant to future soil decision support but differ from the present rule-label classification task [14].

The supplied irrigation article models water allocation under uncertainty and dynamic regulation [15]. Its value to this project is conceptual: reliable decisions must account for changing conditions. The Rosario Dataset v2 provides multimodal agricultural robot observations under challenging field conditions [16]. Duckett et al. discuss agricultural robotics and the research needed for practical deployment [17]. These sources support a staged evaluation of positioning, sensing and intervention rather than treating a laboratory prediction as an operational success.

For learning methods, Random Forest and Isolation Forest provide the foundations of Component 1's exploratory anomaly pipeline [18], [19]. EfficientNet supplies the image backbone for Component 2 [20], while LSTM and XGBoost underpin Component 4's hybrid classifier [21], [22]. Barbedo's plant disease study concerns dataset size and variety, making independent and representative imagery central to evaluation [23]. Component 3 uses clustering measures whose purpose is internal grouping assessment: silhouette, Davies–Bouldin and Calinski–Harabasz [24],

[25], [26]. These outcomes are not interchangeable with diagnostic accuracy or field boundary correctness.

1.3 Research Gap

The research gap identified in the supplied evidence is the incomplete connection between validated component outputs and reliable field actions. The navigation component currently demonstrates an external soil anomaly experiment, rather than measured GNSS reliability or irrigation benefit. Crop monitoring includes a poorly performing eight-category classifier, but no separately evaluated stress or growth model. Zone mapping demonstrates two spectral clusters without verified chilli field polygons. Soil assessment reproduces a constructed scoring rule using one-timestep inputs rather than forecasting future observations [7], [8], [9], [10].

These gaps identify the next empirical contributions: independently labelled local observations, calibrated sensors, coordinate-preserving fusion and supervised operational trials. Leakage and reproducibility research reinforces the need to protect evaluation partitions [27]. The project should therefore demonstrate that each downstream decision retains the uncertainty and validity limits of its upstream evidence. This is a gap within the current project record, not a claim that integrated agricultural systems are absent from the wider literature.

1.4 Research Problem

The overall problem is how an autonomous ground rover can obtain reliable chilli crop and soil observations, translate them into location-aware recommendations and execute bounded actions under changing field conditions. Solving this problem requires agreement among sensing, image interpretation, spatial planning and soil assessment. A failure in one stage can affect all later stages: an incorrect image label can influence a treatment zone, while an unreliable position can direct a correct plan to the wrong location.

The report therefore addresses two related questions. First, what do the available experiments establish about the four analytical components? Second, what additional evidence is required before their combined outputs can support irrigation, seeding or treatment operations? These questions keep model evaluation, software integration and physical rover performance as distinct dimensions of the group research.

1.5 Aim and Objectives

Main objective:
To develop and evaluate an integrated Smart Chilli Crop Intelligence System that uses an autonomous ground rover to support reliable navigation, crop monitoring, field zone planning and soil-informed management.

Table 1.1 shows the corrected component allocation. The specific objectives in Table 1.2 cover analytical development, integration and field evaluation, with progress assessed against the supplied evidence.

Table 1.1: Component allocation

Component	Student	Role
1	IT22112682 Arachci B N S	Intelligent Navigation and Sensor-Based Irrigation Control
2	IT22108722 KULASEKARA D K M	AI-Based Chilli Crop Disease, Stress and Growth Monitoring
3	IT22276032 S G M B Subasinghe	Field Zone Mapping and Autonomous Seeding and Treatment Planning
4	IT22192950 S.N.R Jayawardhana	Soil Health and Soil Intelligence

Table 1.2: Specific research objectives

No.	Objective
1	Define the shared sensing, location and data quality requirements.
2	Develop and evaluate a sensor anomaly and irrigation decision workflow.
3	Develop and evaluate chilli crop image monitoring.
4	Construct and assess field grouping and zone planning methods.
5	Develop and assess soil classification and future temporal intelligence.
6	Implement compatible records and interfaces across the four components.
7	Verify dashboard presentation using genuine inference outputs.
8	Evaluate supervised navigation and field interventions with measured outcomes.

1.6 Scope and Significance

The scope includes the four component proposals, their supplied notebooks, image outputs, model metadata, shared pipeline plan and application archive. It covers the implemented offline workflows and the intended integration architecture. The current sources do not substantiate measured water savings, seed placement accuracy, treatment effectiveness, yield improvement, verified growth prediction or genuine soil forecasting. These outcomes are discussed as research requirements rather than completed results.

The significance of the project lies in bringing several observation types into a common management workflow for chilli cultivation. A shared record can reduce the separation between crop images, soil readings and field locations. Its practical value depends on validated local evidence and understandable recommendations. Maintaining observation provenance and uncertainty also makes the research easier to audit, revise and extend across seasons.

1.7 Report Organisation

Chapter 1 introduces the project, literature, research gap, problem, objectives and scope. Chapter 2 explains the methodology, architecture, four component pipelines, integration, commercialisation considerations and testing plan. Chapter 3 presents the recorded results, discusses dashboard and integration evidence, evaluates objective coverage and summarises student responsibilities. Chapter 4 concludes the report and sets out limitations and future work. References use IEEE numbered citations, while the front matter lists the report's figures, tables and abbreviations.

2. METHODOLOGY

2.1 Research Methodology Overview

The group adopts an iterative engineering methodology supported by quantitative evaluation. Each component begins with a defined input and task, implements a reproducible pipeline, evaluates its errors and specifies an integration contract. The combined system then requires staged tests, moving from offline inputs to software integration and supervised rover operation. This sequence separates evidence of model behaviour from evidence of a working field system.

The present draft is an evidence-based synthesis of the supplied artefacts. Stored notebook outputs are reported as results of the recorded runs; the models have not been retrained for this report. The application code is inspected as a software prototype, without claiming that its routes or database were exercised in a live deployment. Research plans and literature inform the next tests, but neither substitutes for measurements. Dataset versions, feature order, class mapping, preprocessing and model identifiers should be retained for reproducibility [28].

Preprocessing should be fitted on development data and applied unchanged to evaluation data [29]. Time-dependent observations need chronological partitions, while related images need grouped partitions [30], [31]. Thresholds should be selected using validation data and held fixed for testing. Independent field assessments are essential where the current labels are derived from a rule or an unsupervised model.

2.2 Overall System Architecture

The intended system has four connected analytical services and a shared observation store. Component 1 supplies movement context, sensor readings and irrigation control. Component 2 associates images with crop category predictions. Component 4 interprets soil inputs, and Component 3 combines spatial evidence into zone plans. The dashboard provides a review interface, with recommendations returned to the rover through an explicit command pathway [11]. Figure 2.1 represents this intended architecture rather than a verified live deployment.

The supplied software implementation is a Flask application connected to MongoDB, with users, sensor_logs, disease_logs, field_zones and soil_health collections [12]. Flask and PyMongo documentation support the intended application and database layer [32], [33]. The archive seeds example records when collections are empty. It does not demonstrate model loading, camera ingestion, live sensor acquisition, polygon generation or actuator execution. Those interfaces are additional implementation requirements.

An observation record should preserve its identifier, capture timestamp, field and zone context, coordinates, units, quality flags, source and model version. A separate task record should include action type, requested location, expiry, approval status and acknowledgement. This separation prevents a displayed prediction from being mistaken for a completed physical action.

Table 2.1: Architecture layers and implementation status

Layer	Intended function	Evidence status
Rover acquisition	Position, images and soil measurements	Proposed in component reports; field logs not supplied
Offline analysis	Four task-specific pipelines	Code and saved outputs supplied
Shared data	Versioned observation and task records	MongoDB collections supplied; live fusion unverified
Dashboard	Review and component summaries	Flask prototype with seeded examples
Control	Reviewed irrigation, seeding and treatment	No supplied end-to-end actuator test log

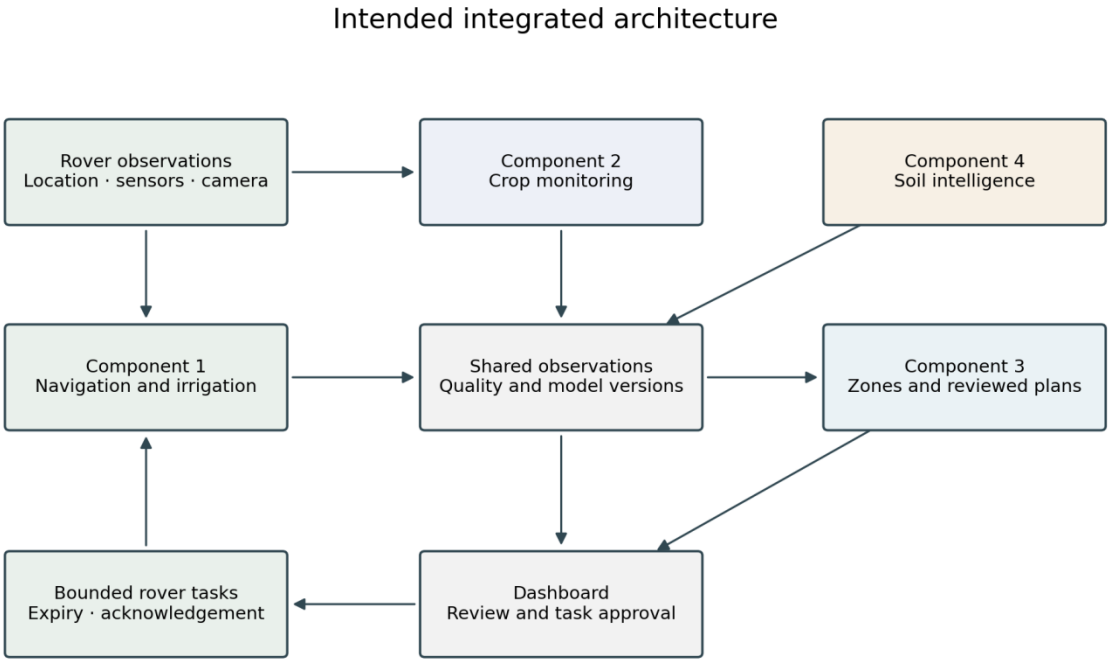


Figure 2.1: Intended integrated system architecture

2.3 Component 1: Intelligent Navigation and Sensor-Based Irrigation Control

Component Methodology

The component follows an iterative engineering methodology: define sensing and navigation requirements, implement an offline model, evaluate error behaviour, connect structured outputs and conduct controlled rover tests. The supplied notebook represents the offline experiment [7]. The next stage should retain separate development, calibration and field evaluation datasets.

Actuator tests must progress from simulation to supervised operation, with logs recording why each irrigation command was accepted, rejected or interrupted.

Dataset Description

The notebook downloads the field-scale soil moisture dataset identified as sathyanarayanrao89/soil-moisture-data-from-field-scale-sensor-network and selects Daily/CAF003.txt [7]. It loads 3,346 rows and 12 original columns. Dates extend from 20 April 2007 to 16 June 2016. Five moisture and five temperature channels describe depths from 30 to 150 cm. These observations are an external sensing benchmark; the supplied evidence does not establish that they were collected in Sri Lankan chilli fields.

Data Preprocessing

Column names are standardised, dates sorted and the location identifier removed before modelling. Missing sensor values are forward-filled and then back-filled; moisture is clipped to 0–1 and temperature to −10–60 °C [7]. Several channels contain more than 1,300 missing observations. Although the comment refers to short gaps, the code does not impose a maximum fill duration. Long missing intervals can therefore become artificial plateaus, while backward filling can introduce information from future observations. The revised pipeline should retain missingness flags and reject gaps that exceed an explicit duration.

Feature Extraction

The feature set contains ten raw channels, seven-day rolling means and standard deviations for each channel, a surface-to-deep moisture gradient, a day-to-day surface moisture change, day of year and month. This produces 34 model inputs [7]. Rolling statistics represent recent behaviour, while depth gradients compare the soil profile. At deployment, these features require a stored history buffer; one current reading cannot reproduce a seven-day statistic. Calendar features also require testing against seasonal shortcut learning.

Model Selection and Training

Isolation Forest uses 200 estimators and a contamination setting of 0.05, producing 168 anomaly labels and 3,178 normal labels. Random Forest uses 300 trees, maximum depth 12, minimum leaf size three and balanced class weights. The model is trained on the first 2,676 records and tested on the last 670 [7]. However, scaling and pseudo-label generation occur before partitioning. A stricter experiment should fit all learned transformations and the anomaly labeller on development data only and evaluate against independently annotated events.

Prediction Method

Inference must apply the saved feature order and scaler to a complete feature vector, then return a class and anomaly probability with timestamp and quality flags. The notebook exports the model, scaler and feature list [7]. An anomaly prediction should initiate inspection or a controlled fallback, not automatically trigger irrigation. The irrigation rule layer should check calibrated moisture limits, recent watering, pump status and location confidence. Proposed GNSS outputs require a separate trained and tested model.

System Architecture and Integration

The planned chain links rover sensing to quality assessment, feature generation, prediction storage and supervised irrigation control. A common record should contain observation_id, timestamp, location, units, quality status and model version. Sensor records feed Components 3 and 4; irrigation requests return with a command identifier and expiry time. The supplied Flask application provides MongoDB collections and dashboard routes, but no demonstrated live model-loading and actuator execution path [12]. This distinction should remain visible in the final implementation record.

Method foundations

Random Forest and Isolation Forest are implemented through their documented scikit-learn interfaces [34], [35]. Missing-data operations should preserve gap duration and quality status [36]. Standardisation parameters must be fitted on development data [37].

Table 2.2: Component 1 dataset and model specification

Item	Recorded specification
Dataset	CAF003 daily file; 3,346 records; 20 Apr. 2007–16 Jun. 2016
Raw inputs	Five moisture and five temperature depth channels
Engineered inputs	34 features including seven-day statistics, gradient, delta and calendar
Label source	Isolation Forest; 200 estimators; contamination 0.05
Classifier	Random Forest; 300 trees; depth 12; minimum leaf size 3
Split	2,676 chronological train; 670 test
Target provenance	168 pseudo-labelled anomalies; not expert-confirmed faults

2.4 Component 2: AI-Based Chilli Crop Disease, Stress and Growth Monitoring

Component Methodology

The methodology comprises dataset audit, image quality checking, augmentation, transfer learning, error analysis and integration testing. The available notebook documents an initial training run rather than a validated field deployment [8]. Development should begin with a small reproducible baseline and visual inspection of transformed inputs. Subsequent experiments should compare corrected preprocessing, class-balanced training and alternative lightweight models on the same fixed evaluation partition.

Dataset Description

The notebook downloads ravindubandara3002/chilli-plant-diseases-dataset and uses the supplied train, valid and test directories [8]. There are 6,163 training images, 479 validation images and 113 test images, with no corrupt files reported by the image verification routine. The class mapping is preserved in the supplied class_names.json. Dataset authorship, augmentation ancestry and farm provenance are not established by folder counts, so these partitions need an independent leakage audit.

Data Preprocessing

Images are resized to 224 × 224 pixels. Training preprocessing applies rescaling by 1/255, rotation, translation, shear, zoom, horizontal flipping and brightness variation. Validation and test generators apply rescaling without augmentation [8]. EfficientNet input handling should be checked against the installed model implementation because a model with internal rescaling may require a different external range. This is a diagnostic hypothesis, not an established explanation of the observed errors. Before retraining, display tensors and verify pixel ranges at the backbone input.

Feature Extraction

EfficientNet-B0 provides learned convolutional features, which are passed to the classification head. The saved code supports an eight-category output rather than a multi-label representation [8]. Consequently, a plant exhibiting several conditions is forced into one class. Field inference should preserve the full probability vector and avoid interpreting a softmax value as calibrated diagnostic certainty. A separate severity or growth head would require additional labels and an independent evaluation.

Model Selection and Training

The notebook defines image size 224, batch size 32, an initial learning rate of 10^{-4} and a 30-epoch setting [8]. It uses transfer learning followed by fine-tuning, with learning curves saved for review. The observed outcome makes an optimisation audit necessary. A controlled comparison should first test whether the model can learn a small verified subset, then examine gradients, trainable layers, class weighting and preprocessing consistency. Improvement must be measured on a held-out partition, not inferred from training accuracy.

Prediction Method

The inference path should check image readability and capture quality, apply the same resizing and pixel treatment used in training, obtain the probability vector and map indices using the exported class order. The output record should contain class, probability distribution, uncertainty status and source image. A review threshold should be calibrated on validation data. Images outside the training distribution, poor-quality captures and ambiguous symptom combinations should be routed for inspection rather than forced into a confident management instruction.

System Architecture and Integration

The proposed architecture connects camera capture, quality checking, preprocessing, disease inference and geotagged storage. Components 3 and 4 receive observations with freshness and

confidence information, while the dashboard presents the image and associated label [2], [11]. The application archive includes disease_logs and display routes but no demonstrated call to the exported image model [12]. Its seeded disease examples include Powdery Mildew, which is absent from the notebook class mapping. Production integration must reconcile these schemas.

Training controls and future extensions

Official EfficientNet documentation states the expected preprocessing range, which must be checked against the notebook's external rescaling [38]. Transfer learning requires an explicit freeze and fine-tuning schedule [39]. Adam provides the optimisation foundation [40], and early stopping provides a validation-based control rather than a guarantee of generalisation [41]. Grad-CAM can support future visual inspection of attention regions [42]. MobileNets and MobileNetV2 are candidate lightweight comparisons, not implemented results of this project [43], [44]. The exported class mapping must remain authoritative for inference [45].

Table 2.3: Component 2 image split counts

Category	Train	Valid	Test
Whitefly	500	10	10
Yellowish	500	10	10
Anthracnos	430	54	43
Damping_Off	168	10	10
Leaf_Curl_Virus	500	10	10
Leaf_Spot	500	10	10
Veinal_Mottle_Virus	2,000	10	10
healthy	1,565	365	10
Total	6,163	479	113

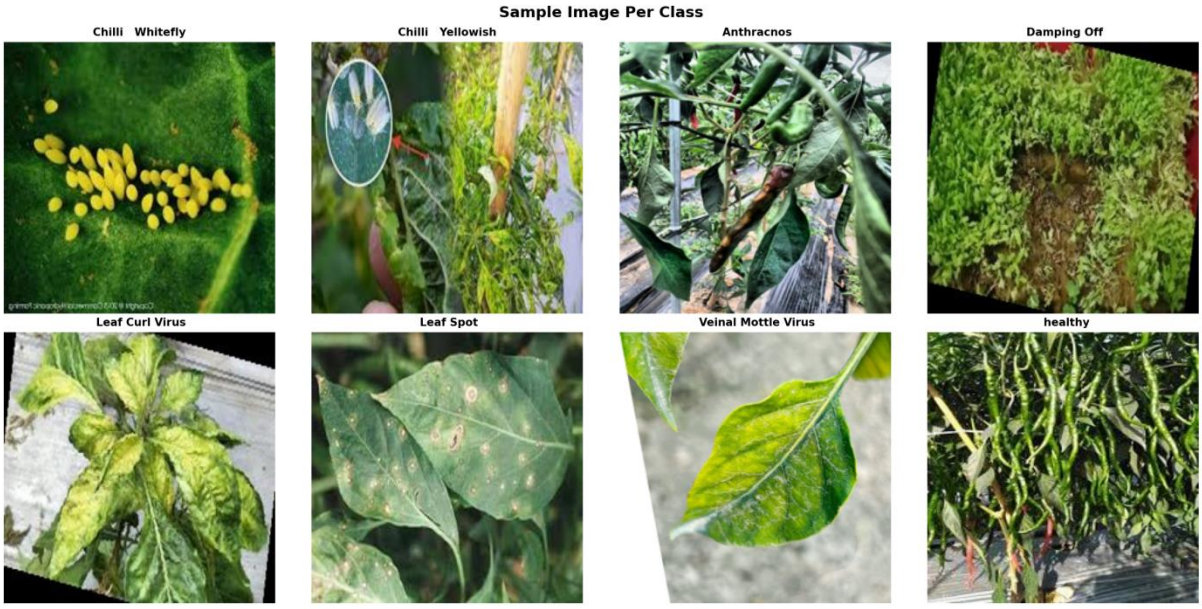


Figure 2.2: Sample chilli category images

2.5 Component 3: Field Zone Mapping and Autonomous Seeding and Treatment Planning

Component Methodology

The methodology starts with a reproducible clustering benchmark, then adds spatial registration and agronomic interpretation. The initial experiment explores six spectral inputs, compares candidate K values and profiles the chosen clusters [9]. A subsequent field phase should collect matched location, soil and crop observations; derive management-specific features; generate mapped zones; obtain expert review; and evaluate supervised rover tasks. Each stage should retain its inputs, parameters and output version.

Dataset Description

The notebook downloads abdulahadiltaf/sentinel-2-multispectral-crop-and-land-cover-dataset [9]. The clean feature matrix contains 120,130 records and six columns. The selected matrix removes other columns and therefore does not itself preserve a geospatial schema. The dataset is an external crop and land-cover benchmark; the available source does not establish a Sri Lankan chilli field origin. Local management claims require additional coordinate and crop validation.

Data Preprocessing

The implementation selects available spectral and index columns, removes missing records and applies its cleaning routine before standardisation [9]. The resulting matrix is transformed with StandardScaler because K-Means is distance-based. A local spatial pipeline should additionally preserve source identifiers, coordinate reference system, acquisition time and quality flags. Coordinate transformation and interpolation must be explicit operations, with uncertainty retained for areas lacking direct observations.

Feature Extraction

The six implemented inputs are NIR, SWIR, NDVI, NDWI, EVI and SAVI. Principal component analysis is used for visualisation of the scaled feature space; it is not a surveyed field map [9]. Future fusion can include moisture, reviewed disease evidence and soil condition, but those features must be calibrated and temporally aligned. Weights proposed in the design document should be treated as hypotheses requiring sensitivity analysis rather than established optimal values.

Model Selection and Training

The notebook samples up to 20,000 records for a search over k = 2–12 and estimates silhouette on a sample of 5,000. The highest search silhouette is 0.5421 at k = 2 [9]. The final K-Means uses k-means++ initialisation, 15 initialisations, a maximum of 500 iterations and random state 42. It fits the full scaled matrix and converges in four iterations. This selection reflects the supplied spectral benchmark, not a requirement that every field must have two zones.

Prediction Method

For a new observation, the pipeline should validate feature names and scales, apply the exported scaler and assign the nearest centroid. The numerical cluster identifier is then interpreted using a stored centroid profile. Distance to the nearest centroid should be retained as an uncertainty cue because a distant sample may be outside the fitted distribution. A planning layer should combine cluster evidence with crop stage, field constraints and reviewed agronomic rules before generating a seeding or treatment task.

System Architecture and Integration

The planned architecture ingests timestamped observations from all components, aligns them spatially, constructs a feature grid, delineates zones and produces a reviewed plan [3], [11]. Zone records should store polygon geometry, coordinate reference system, contributing observations, model version and action status. The supplied application stores field_zones summaries but does not demonstrate polygon generation or live clustering inference [12]. Operational tasks require a separate command record and acknowledgement from Component 1.

Clustering controls

The documented KMeans interface defines fitting and assignment of numerical labels [46]. Silhouette, Davies–Bouldin and Calinski–Harabasz implementations provide complementary internal measures [47], [48], [49]. They should be supplemented with spatial and agronomic validation.

Table 2.4: Component 3 clustering specification

Item	Recorded specification
Clean matrix	120,130 records × six features
Features	NIR, SWIR, NDVI, NDWI, EVI, SAVI
Scaling	StandardScaler
K search	k = 2–12; up to 20,000 records; 5,000 silhouette sample
Final fit	k = 2; k-means++; n_init = 15; max_iter = 500
Outputs	Cluster assignment, centroids, model and visualisations
Spatial limitation	Six-feature selected matrix does not preserve mapped geometry

2.6 Component 4: Soil Health and Soil Intelligence

Component Methodology

The methodology begins with a transparent baseline classification experiment and extends toward independently validated soil intelligence. The supplied notebook documents ingestion, score construction, filtering, learning and evaluation [10]. A field phase should record sampling depth, sensor calibration, units, location and time, with reference soil assessments on a subset. Temporal development should use contiguous histories from the same zone and retain a later-period holdout for forecasting evaluation.

Dataset Description

The notebook downloads javakhan/crops-npk-data-set and reads sensor_Crop_Dataset (1).csv [10]. It contains 20,000 rows and ten original columns: seven numerical inputs plus Crop, Soil_Type and Variety. The implemented model uses Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH_Value and Rainfall. The data preview contains several crop types, so the dataset should not be described as a verified chilli-only collection. Units and original collection provenance need confirmation before sensor integration.

Data Preprocessing

The notebook reports no missing values in the initial table. It generates target labels, encodes categorical descriptors and filters each numerical feature using sequential 1st and 99th percentile limits, then removes duplicates [10]. Despite an IQR comment, the implemented filter is percentile-based. The resulting table has 17,356 rows and 18 columns. StandardScaler is fitted before splitting, which should be corrected by training-only fitting. Filtering can also remove conditions important for critical-risk detection, so excluded extremes need a separate audit.

Feature Extraction

The seven numerical inputs form the model vector. The soil score is computed as 0.5 times normalised NPK sum, plus 0.3 times inverse normalised absolute deviation from pH 6.5, plus 0.2 times normalised humidity [10]. Scores below 0.40 are Poor, scores below 0.65 are Moderate and remaining scores are Healthy. Dataset-wide maxima set the normalisation terms. The LSTM generates a 64-dimensional representation used by XGBoost. Both the rule and fitted transforms must be versioned because changing the reference dataset can change the score scale.

Model Selection and Training

The supplied split contains 12,539 training, 2,213 validation and 2,604 test cases. Stratified random splitting allocates 15% to testing and 15% of the remaining data to validation [10]. The LSTM uses 128- and 64-unit layers with regularisation and a three-class output. Its input shape is (samples, 1, 7). Training allows up to 80 epochs with validation-based callbacks. The hybrid classifier uses the 64 learned embedding values together with seven scaled original inputs, producing 71 features rather than a genuine observation sequence.

Prediction Method

A new sample must contain all seven fields in the order preserved by metadata.json [50]. It is scaled, reshaped to one timestep, passed through the trained representation, concatenated with the seven scaled original values and classified by XGBoost. The label encoder maps outputs to Healthy, Moderate and Poor. A prediction should be described as agreement with the project score definition, with missing-input and out-of-range checks. It should not be advertised as a forecast or converted into a fertiliser rate without independent agronomic evidence.

System Architecture and Integration

The planned soil intelligence interface receives timestamped, located sensor records, combines them with crop evidence and returns versioned assessments to zone mapping and the dashboard [4], [11]. The implemented model expects NPK, temperature, humidity, pH and rainfall; Component 1's external notebook provides moisture and temperature at several depths. These schemas are not directly interchangeable. Integration must identify the actual source of each required variable and handle unavailable inputs without fabrication. The supplied application stores demonstration soil records but does not show a live inference call [12].

Hybrid inference controls

The LSTM interface expects an explicitly defined sequence dimension [51]. NumPy hstack documents the horizontal concatenation used to form the 71-dimensional hybrid vector [52]. XGBoost input order and parameters should be retained with the exported model [53]. The stored metadata provides the seven-field and three-label mapping [50].

Table 2.5: Component 4 dataset and hybrid specification

Item	Recorded specification
Data	20,000 original rows; 17,356 retained
Features	Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH_Value, Rainfall
Target	Rule-derived Healthy, Moderate and Poor labels
Partitions	12,539 train; 2,213 validation; 2,604 test
LSTM input	One timestep × seven features per case
Representation	64-dimensional embedding
XGBoost input	64 embeddings + seven scaled original fields = 71 features
Hybrid parameters	400 estimators; max_depth 6; learning_rate 0.08; subsample and colsample 0.85

2.7 Integrated Data Flow and System Interaction

The common data flow begins with observation acquisition, followed by quality checks, component-specific preprocessing and inference. Records are aligned by location and capture time before fusion. Components 2 and 4 supply assessments to Component 3, which creates a reviewed spatial plan. Component 1 receives only tasks compatible with current positioning quality and actuator status. Figure 2.3 shows the proposed decision and acknowledgement cycle.

The sources expose important schema differences. Component 1's notebook uses moisture and temperature channels at five depths; Component 4 expects NPK, temperature, humidity, pH and rainfall. Component 2's exported class list has eight categories, while the dashboard examples include a label outside that vocabulary. Component 3 fits two clusters, but the dashboard seeds four named zones [7], [8], [9], [10], [12], [50], [45]. Integration must resolve these differences explicitly rather than assigning values or labels that were not measured.

Geographic records require an identified coordinate reference system, and any transformation must preserve traceability. GeoPandas and pyproj document the distinction between assigning and transforming coordinate systems [54], [55]. Spectral clustering visualisations cannot supply missing field geometries. Weather services such as NASA POWER can provide contextual data, but retrieval dates, spatial resolution and observation periods must be recorded, and weather inputs must not be presented as rover measurements [56].

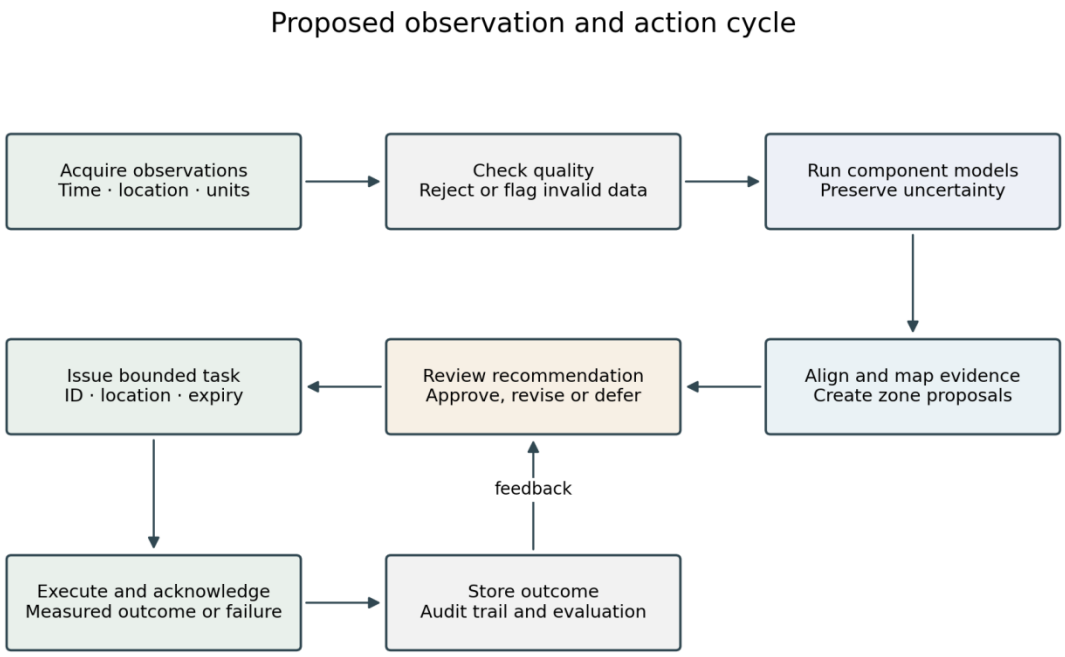


Figure 2.3: Proposed observation and action cycle

2.8 Commercialisation Aspect of the Product

A potential product would combine a supervised rover inspection service with a dashboard that organises plant images, sensor quality and zone-level observations. Initial users could include research farms, extension services and growers who need repeated monitoring records. This is a proposed commercialisation pathway, not a market study or evidence of willingness to pay. Interviews and observed workflow trials are needed before selecting a business model.

A staged offering should begin with monitoring and review. Automated interventions should be introduced only after the relevant models, sensors and control functions meet agreed validation criteria. Costs would include the rover, sensor calibration, batteries, maintenance, data handling, model support and operator training. A pilot should measure inspection time, usable data coverage, maintenance demands and user understanding. Claimed savings or productivity benefits should be based on a defined comparison rather than literature outcomes from other systems.

Data ownership, export, access roles and service continuity should form part of product design. Local language summaries may improve usability, but they should preserve the uncertainty of the original assessment. The prototype's default credentials and hard-coded secret are development settings that need replacement before external use [12]. No selling price, revenue forecast or commercial readiness claim is established by the supplied files.

2.9 Testing and Implementation Plan

Testing is organised into data validity, model evaluation, application integration and supervised physical operation. Table 2.6 defines proposed checks and the evidence required for completion. The plan should be executed with frozen datasets and versioned software so that failures can be reproduced. Successful offline model export does not complete an application test, and a populated dashboard does not complete a rover trial.

Machine learning tests should compare meaningful baselines. Component 1 needs anomaly recall and event detection delay; Component 2 needs per-class recall, macro F1 and calibration; Component 3 needs cluster stability and independent spatial assessment; Component 4 needs agreement with independent soil references and temporal baselines when forecasting is implemented. Classification reports summarise per-class outcomes [57], but metric choice must reflect the actual task. Precision–recall analysis is especially relevant for rare anomaly events [58].

Integration tests should use fixed known inputs and compare dashboard records with offline predictions. Fault tests should include missing variables, invalid units, stale positions, unknown labels, interrupted connectivity and duplicated commands. Supervised actuation trials should independently measure navigation error, irrigation volume, seed placement and treatment coverage. All unexecuted checks remain pending in this draft.

Table 2.6: Testing and implementation plan

Test area	Method and acceptance evidence	Current status
Sensor calibration	Compare units, depth and readings with reference measurements	Pending local evidence
C1 anomaly evaluation	Reviewed event labels; recall, false alarms and delay	Recorded prototype inadequate
C2 classification	Fixed independent images; macro F1 and class recall	Recorded prototype inadequate
C3 zones	Repeat stability, surveyed geometry and expert review	Internal clustering only
C4 soil validity	Independent references; exact-rule and simple baselines	Constructed-label agreement only
Application inference	Known input agrees with offline output and stored record	Not executed in supplied logs
Task controls	Expiry, duplicates, acknowledgement and failure cases	Not executed in supplied logs
Field interventions	Surveyed navigation, water, seed and treatment outcomes	No supplied measured trial

3. RESULTS AND DISCUSSION

3.1 Results Overview

The available results represent four offline experiments at different levels of maturity. Component 1 reports high aggregate accuracy but no successful anomaly detection in its test positives. Component 2 reports low image recognition performance. Component 3 produces statistically separated spectral groups without demonstrated field polygons. Component 4 reports high agreement with rule-generated classes, but lacks independently measured targets and temporal forecasting [7], [8], [9], [10]. Table 3.1 summarises these outcomes without treating them as directly comparable accuracies.

The main group finding is that useful development artefacts exist, but end-to-end operational validity remains unproven. This does not invalidate the architectural aim. It identifies the tests and corrections required to turn isolated experiments into reliable field intelligence. The figures in this chapter are taken from supplied notebook outputs; numerical comparisons explicitly identified as calculated are derived from those outputs.

Table 3.1: Recorded component results

Component	Recorded outcome	Interpretation
C1	95.82% calculated accuracy; anomaly recall 0.00	Misses all nine pseudo-labelled test anomalies
C2	13.27% accuracy; macro F1 ≈ 0.05	Current image classifier inadequate
C3	Silhouette 0.5423; two clusters	Feature-space separation; no validated field polygons
C4	LSTM 99.12%; hybrid 99.46%	Agreement with constructed classes; no future forecasting

3.2 Navigation and Sensor Based Irrigation Results

The supplied Component 1 notebook retains 3,346 CAF003 daily observations and uses 34 engineered features. It assigns 168 anomalies through Isolation Forest and trains Random Forest on the first 2,676 cases, with 670 cases reserved for testing [7]. The confusion matrix contains 642 correctly predicted normal cases, 19 normal cases flagged as anomalies, nine missed anomalies and zero correctly identified anomalies. Consequently, accuracy is 642/670 = 95.82%, matching the report's rounded 0.96, while anomaly recall and precision are both zero.

An always-normal test baseline would obtain 661/670 = 98.66% accuracy, calculated from the test support. The trained model therefore performs below that baseline on accuracy and detects none

of the nine test events. This shows why aggregate accuracy is not an adequate success criterion for the anomaly task. A revised experiment needs reviewed events, train-only preprocessing, threshold calibration and detection-delay analysis.

No supplied source provides measured GNSS navigation error, collision avoidance performance, irrigation volume or water savings for the current rover. These functions remain part of the proposed component scope [1], [11]. The irrigation literature informs the design of uncertainty-aware decisions [15], but its results must not be reported as project outcomes.

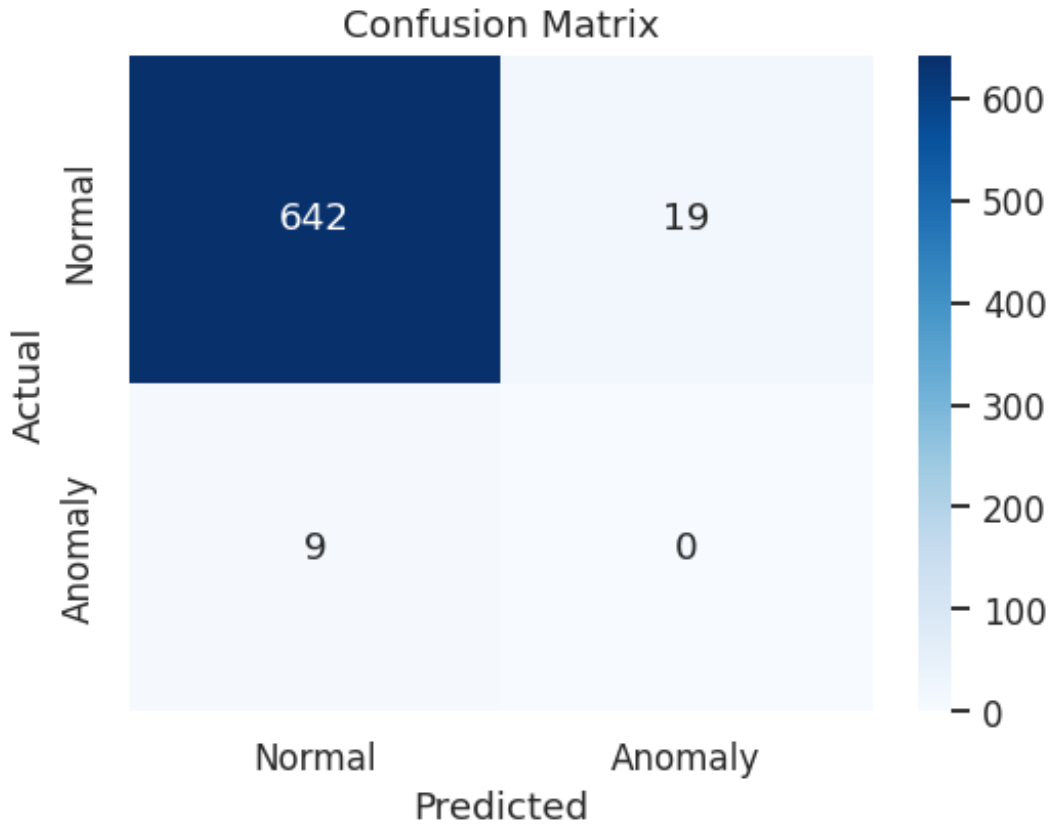


Figure 3.1: Component 1 test confusion matrix

3.3 Crop Monitoring and Field Zone Mapping Results

Component 2 uses 6,163 training, 479 validation and 113 test images across eight categories. The saved test accuracy is 13.27%, or 15 correct predictions. Six categories have zero recall; veinal mottle virus recall is 1.00 on ten cases and healthy recall is 0.50 on ten cases. Their precision values are approximately 0.14 and 0.12 [8]. Figure 3.2 displays the learning curves, and Figure 3.3 shows the concentration of predictions in a small part of the label space.

A uniform eight-class reference is 12.5%, although this does not account for the imbalanced test set. An always-anthracnose predictor would achieve 43/113 = 38.05%, calculated from the largest

test category. The present classifier therefore falls below a simple majority-class baseline. The cause is not established by the outputs. Preprocessing consistency, label mapping, optimisation and dataset independence need controlled investigation. The supplied evidence does not establish separate stress, growth-stage or harvest-time performance.

Component 3 fits two spectral clusters after cleaning 120,130 records. Its final silhouette is 0.5423, Davies–Bouldin index 0.6799 and Calinski–Harabasz score 215,201.56 [9]. The groups contain 76,291 and 43,839 records and have mean NDVI values of 0.6989 and 0.2537. These results support relative spectral separation. They do not establish contiguous chilli field zones, disease classes, mapped area percentages or autonomous seeding and treatment performance. Figure 3.5 is a principal component projection rather than a geographic map.



Figure 3.2: Crop classifier training and validation curves

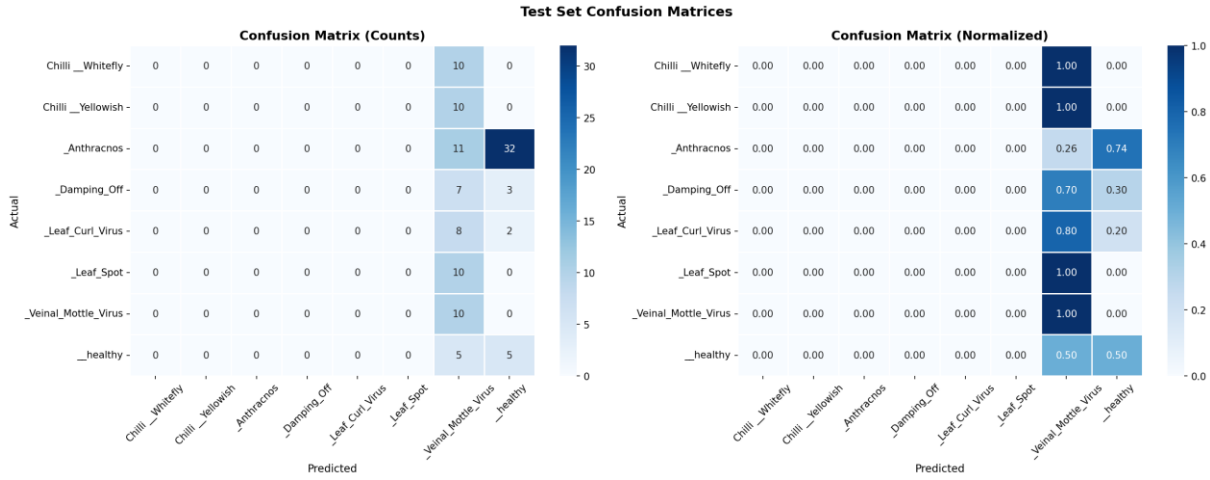


Figure 3.3: Crop classifier test confusion matrices

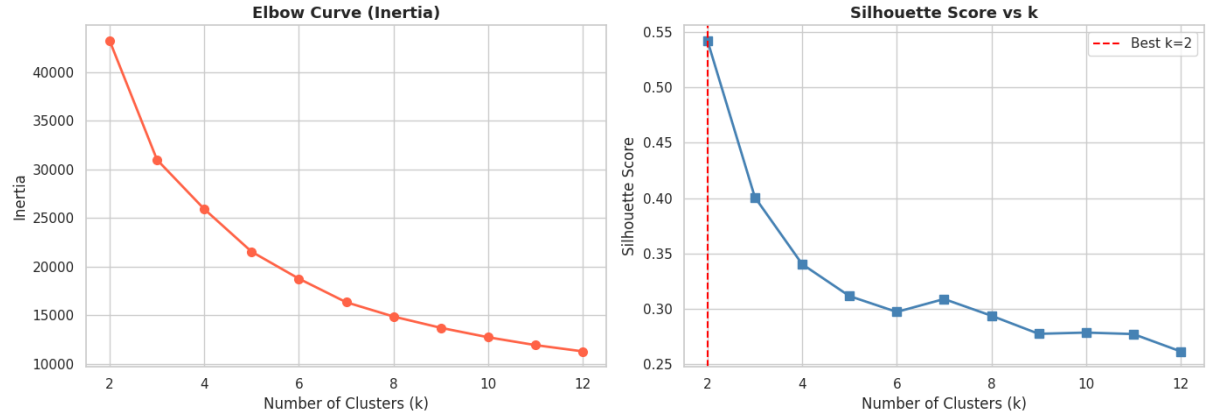


Figure 3.4: Candidate cluster number evaluation

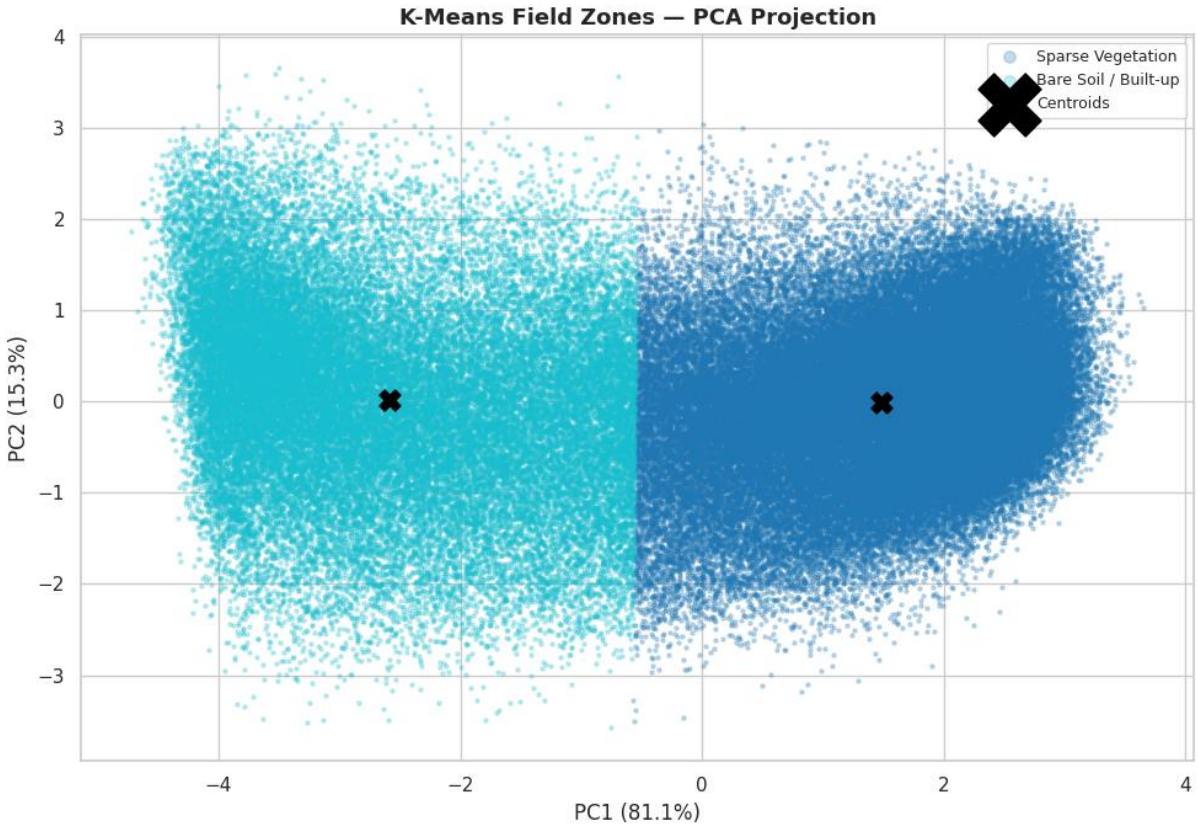


Figure 3.5: Spectral clusters in principal component space

3.4 Machine Learning Detection and Prediction Results

Component 4 retains 17,356 records from an initial 20,000 and evaluates 2,604 test cases. The stored accuracies are 99.1167% for LSTM and 99.4624% for LSTM plus XGBoost, an improvement of about 0.35 percentage points [10], [50]. The hybrid uses 64 learned embedding values concatenated with seven scaled original inputs. Its strong agreement is measured against health labels generated from NPK sum, pH proximity and humidity.

The current experiment is a static classification task. Each LSTM sample has one timestep and seven features, so there is no multi-observation temporal history within a sample. The model does not predict a later soil measurement. The exact labelling rule is an essential baseline because it reproduces the constructed classes by definition. Independent laboratory or expert labels are needed to assess agronomic validity. Figures 3.6 and 3.7 document the recorded learning behaviour and classification matrices.

Across components, metric interpretation must follow target provenance. Component 1 measures agreement with unsupervised pseudo-labels, Component 2 with image folder categories, Component 3 with internal cluster structure and Component 4 with a deterministic score. Their outcomes cannot be averaged into a meaningful overall system accuracy. Confidence calibration

also needs a separate evaluation; a high prediction probability is not automatically a reliable confidence estimate [59], [60].

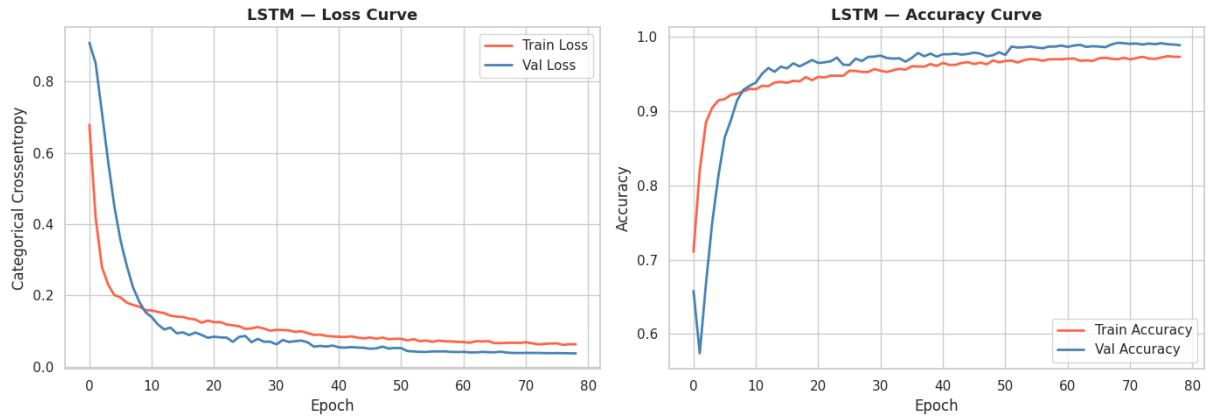


Figure 3.6: Soil classifier LSTM learning curves

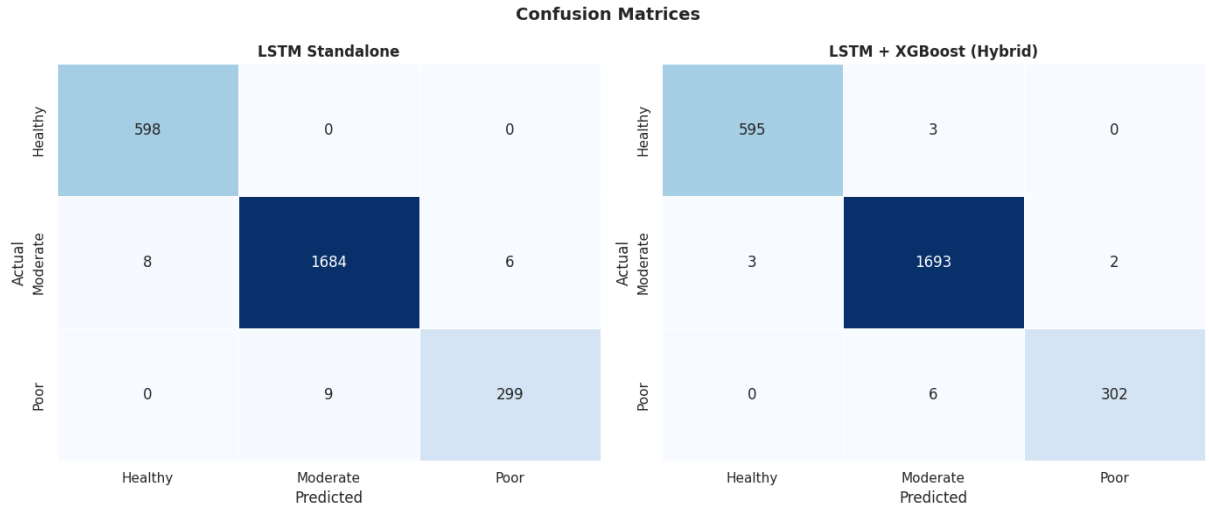


Figure 3.7: Soil classifier test confusion matrices

3.5 Cloud Dashboard and Alerting Results

The template's cloud dashboard heading is retained, but the supplied archive establishes a locally configured Flask and MongoDB prototype rather than a verified cloud deployment. The README specifies localhost:5000 and a local MongoDB service [12]. The code contains authentication, dashboard views, user management and component summary routes. It seeds sample sensor, disease, zone and soil records to demonstrate presentation. These fixtures are not outputs from the notebook test sets.

The interface can serve as the proposed review layer once genuine inference records are connected. At present, there is no supplied execution evidence for a live model service, push notifications or remote alert delivery. A critical integration mismatch is that a seeded disease example uses Powdery Mildew, absent from the eight-class model mapping. Another mismatch is the four

demonstration zones compared with the two fitted spectral clusters. Table 3.2 records these findings as code inspection results, not passed runtime tests.

Table 3.2: Dashboard code inspection findings

Inspected feature	Source finding	Qualification
Application stack	Flask with local MongoDB configuration	Cloud hosting not established
Authentication and users	Routes and user collection supplied	Runtime and access tests not supplied
Sensor display	Three records seeded when empty	Demonstration values, not trial observations
Disease display	Includes Powdery Mildew fixture	Not in exported eight-class mapping
Zone display	Four seeded named zones	Different from two notebook clusters
Soil display	Healthy, Moderate and Poor examples	Not test predictions
Alert delivery	No supplied delivery trace	Notification success not established

3.6 Integrated System Testing Results

No complete end-to-end test log is supplied for rover sensing, inference, zone planning, dashboard review and actuator acknowledgement. The current evidence supports structural compatibility work and identifies interface gaps, but it does not establish a completed integration trial. Table 3.3 distinguishes inspected artefacts from tests still awaiting execution. This status is intentionally different from the proposed test plan in Chapter 2.

A valid integration result would require the same known observation to remain traceable through every stage. The record should preserve location, time, units, label vocabulary and model version. A reviewed task should be issued once, executed within its limits and acknowledged with an observed outcome. Failures should be logged rather than converted into invented normal measurements. Field tests must then examine whether those outcomes are meaningful for chilli cultivation.

Table 3.3: Integrated evidence and pending verification

Integration area	Available evidence	Result status
Notebook outputs	Four experiments with saved outputs	Available; not retrained in this report
Export metadata	Feature and class maps supplied	Available; inference parity untested
Database schema	Component collections in app.py	Inspected; cross-component fusion untested
Live rover ingestion	Proposals and pipeline plan	Pending implementation evidence
Geographic zone plans	Spectral clustering figures	No validated geographic boundaries
Actuator acknowledgement	Intended architecture	No supplied executed test
End-to-end field benefit	Research aim	No measured integrated outcome

3.7 Discussion Against Research Objectives

The project has achieved partial development of all four analytical components and a shared presentation prototype. The notebooks establish reproducible code and recorded experiment outputs, while the proposals and design document describe the intended combined system. They do not establish achievement of every functional objective. Table 3.4 maps each objective to the available evidence and the remaining work.

The strongest immediate research contribution is the systematic identification of where apparent technical success can be misleading. Component 1's high accuracy masks missed anomalies, Component 2 requires corrective training, Component 3's internal separation does not verify spatial management relevance, and Component 4's high accuracy reflects constructed labels. These findings direct the next experiments toward task validity rather than larger datasets or more complex architectures alone.

Integration should proceed with uncertainty-aware records and a review pathway. Introducing downstream automation before upstream validity is demonstrated would compound errors. The intended rover workflow remains feasible as a research direction, but the final performance claim must be based on local, independently evaluated data and measured physical outcomes.

Table 3.4: Objective coverage and remaining work

Objective area	Evidence assessment	Required next step
Shared requirements	Defined through proposals and architecture	Freeze units, identifiers and quality schema
Navigation and irrigation	Anomaly notebook only; poor positive recall	Reviewed faults and supervised control trials
Crop monitoring	Classifier implemented; low accuracy	Preprocessing audit and independent retraining
Zone planning	Spectral grouping implemented	Local boundaries and agronomic validation
Soil intelligence	Static rule-label classification	Independent labels and real time histories
Cross-component integration	Design plus database prototype	Live inference and schema parity tests
Dashboard verification	Seeded example presentation	Replace fixtures and test real outputs
Operational outcomes	No supplied field test log	Measure physical intervention performance

3.8 Summary of Each Student's Contribution

Table 3.5 records the component responsibilities specified by the corrected allocation. The association between each supplied notebook and its component follows the filenames and task descriptions; it does not establish individual authorship of every code cell or completion of all proposed functions. The final submission should be checked by the group against its actual implementation and contribution records.

Shared responsibilities include interface definitions, integration decisions, evidence management, testing and report review. An individual component output becomes a group contribution when its schema and uncertainty are compatible with the common workflow. The group should retain commit history, test ownership and integration logs to document that work beyond the proposal-level allocation.

Table 3.5: Student component responsibilities and supplied artefacts

Student	Contribution
IT22112682 Arachci B N S	Component 1 responsibility: navigation and irrigation design; associated sensor anomaly pipeline and evaluation requirements.
IT22108722 KULASEKARA D K M	Component 2 responsibility: crop disease, stress and growth design; associated image classification experiment and figures.
IT22276032 S G M B Subasinghe	Component 3 responsibility: field zones and planning design; associated multispectral K-Means experiment.
IT22192950 S.N.R Jayawardhana	Component 4 responsibility: soil health and intelligence design; associated LSTM–XGBoost experiment and metadata.

4. CONCLUSION AND RECOMMENDATIONS

4.1 Overall Conclusion

The Smart Chilli Crop Intelligence System presents a coherent research architecture linking rover mobility, crop observation, spatial planning and soil assessment. The supplied artefacts demonstrate four offline analytical experiments and a shared dashboard prototype. They provide a foundation for further development, but they do not establish a fully validated autonomous chilli management system.

The group results require differentiated conclusions. The anomaly classifier needs revision because it misses every test anomaly. The image classifier is inadequate at its recorded test performance. Spectral clustering provides a useful exploratory grouping result but lacks verified field boundaries and actuation evidence. The hybrid soil classifier accurately reproduces its generated label rule, while genuine future soil prediction remains unimplemented. These conclusions preserve the value of the development work while identifying the evidence needed for the project's operational aims.

4.2 Limitations

The research currently relies on external benchmark datasets with incomplete evidence of local chilli relevance. Component 1 contains extensive imputation and pseudo-labels. Component 2 has imbalanced partitions, limited test support and poor recognition. Component 3's selected feature matrix does not preserve the geometry needed for actual field zones. Component 4 uses derived labels and one-step samples. Full-data preprocessing in some notebooks also limits the independence of reported evaluations.

The application record is based on code inspection and seeded demonstrations, without a supplied live deployment or integrated test log. Hardware performance, connectivity robustness, navigation accuracy, irrigation outcomes, seed placement and treatment coverage are unmeasured in the available files. The missing PP2 presentation cannot be used to fill those gaps. Literature informs design choices but does not validate the project's implementation. These limitations should be resolved through additional empirical work rather than removed from the report wording.

4.3 Recommendations and Future Work

First, rebuild evaluation pipelines around independent partitions. Fit scalers and learned preprocessing on training data, use chronological splits for temporal observations and group related plant images before augmentation. Component 1 needs independently reviewed fault events and precision–recall analysis. Component 2 should audit EfficientNet input ranges, class mapping and optimisation, then compare a simple baseline on a fixed test set.

Second, validate local meaning. Component 3 needs coordinate-preserving field records, independently reviewed zone profiles and boundary stability tests. Component 4 needs calibrated units and laboratory or expert soil references. Compare its hybrid model with direct XGBoost and

the exact score rule. To support forecasting, collect consecutive timestamped histories, define future targets and evaluate against persistence and statistical baselines.

Third, implement the shared interfaces and demonstrate traceability. Replace seeded dashboard examples with actual model outputs, reconcile labels and cluster counts, preserve observation quality and require command acknowledgement. Conduct supervised rover trials that measure navigation error, water volume, seed placement and treatment coverage. Only after these tests should the group evaluate commercial pilots, user benefits or autonomous operation claims. Prioritised actions and evidence requirements are summarised in Table 4.1.

Table 4.1: Prioritised future work

Priority	Action	Completion evidence
1	Correct partitioning and preprocessing	Auditable train-only transforms and fixed holdouts
1	Resolve C1 and C2 performance failures	Independent labels, baseline comparisons and per-class results
1	Verify C4 target validity	Laboratory/expert references and exact-rule comparison
2	Build genuine time-series forecasting	Chronological histories and future-target error analysis
2	Create local field geometries	Surveyed coordinates and stable reviewed zones
2	Connect real inference to dashboard	Known-input parity and traceable storage
3	Run supervised rover tasks	Measured navigation and intervention outcomes
3	Evaluate product pilot	Observed user value, costs and maintainability

REFERENCES

[1] B. N. S. Arachci, “Intelligent navigation and sensor-based irrigation control for chilli cultivation,” project proposal, R26-IT-075, Sri Lanka Institute of Information Technology, Mar. 2026. [Unpublished project document].

[2] D. K. M. Kulasekara, “AI-based chilli crop disease, stress and growth monitoring using a ground rover,” project proposal, R26-IT-075, Sri Lanka Institute of Information Technology, Mar. 2026. [Unpublished project document].

[3] S. G. M. B. Subasinghe, “Field zone mapping and autonomous seeding and treatment planning,” project proposal, R26-IT-075, Sri Lanka Institute of Information Technology, Mar. 2026. [Unpublished project document].

[4] S. N. R. Jayawardhana, “Smart Chilli Crop Intelligence System using an autonomous ground rover,” soil intelligence project proposal, R26-IT-075, Sri Lanka Institute of Information Technology, Mar. 2026. [Unpublished project document].

[5] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, “Machine learning in agriculture: A review,” Sensors, vol. 18, no. 8, Art. no. 2674, 2018, doi: 10.3390/s18082674.

[6] L. Benos, A. C. Tagarakis, G. Dolias, R. Berruto, D. Kateris, and D. Bochtis, “Machine learning in agriculture: A comprehensive updated review,” Sensors, vol. 21, no. 11, Art. no. 3758, 2021, doi: 10.3390/s21113758.

[7] R26-IT-075 project team, “Component 1 anomaly detection,” component_1_anomaly_detection.ipynb, undated. [Unpublished notebook; code and saved outputs].

[8] R26-IT-075 project team, “Chilli disease detection,” chilli_disease_detection.ipynb and associated image outputs, undated. [Unpublished notebook].

[9] R26-IT-075 project team, “Field zone mapping K-Means,” field_zone_mapping_kmeans.ipynb, undated. [Unpublished notebook; code and saved outputs].

[10] R26-IT-075 project team, “Soil health detection using LSTM and XGBoost,” soil_health_detection_lstm_xgboost.ipynb, undated. [Unpublished notebook].

[11] R26-IT-075 project team, “Machine learning pipeline design all 4 components,” ML_Pipelines_SmartChilli.docx, undated. [Unpublished design document].

[12] R26-IT-075 project team, “ChilliRover Intelligence System Flask dashboard,” chilli_app.zip, app.py, requirements.txt and README.md, undated. [Unpublished software prototype].

[13] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” Computers and Electronics in Agriculture, vol. 147, pp. 70–90, 2018, doi: 10.1016/j.compag.2018.02.016.

[14] A. Chlingaryan, S. Sukkarieh, and B. Whelan, “Machine learning approaches for crop yield prediction and nitrogen status estimation in precision agriculture: A review,” Computers and Electronics in Agriculture, vol. 151, pp. 61–69, 2018, doi: 10.1016/j.compag.2018.05.012.

[15] G. Yang et al., “Optimized allocation of irrigation water resources based on uncertainty: Model construction and dynamic regulation mechanism,” Water, vol. 18, no. 5, Art. no. 612, 2026, doi: 10.3390/w18050612.

[16] N. Soncini, J. Cremona, E. Vidal, M. García, G. Castro, and T. Pire, “The Rosario Dataset v2: Multimodal dataset for agricultural robotics,” The International Journal of Robotics Research, 2025, doi: 10.1177/02783649251368909.

[17] T. Duckett et al., “Agricultural robotics: The future of robotic agriculture,” UK-RAS Network White Papers, 2018. [Online]. Available: https://arxiv.org/abs/1806.06762. Accessed: Oct. 11, 2026.

[18] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[19] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in Proc. 8th IEEE International Conference on Data Mining, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[20] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. 36th International Conference on Machine Learning, vol. 97, 2019, pp. 6105–6114. [Online]. Available: https://proceedings.mlr.press/v97/tan19a.html.

[21] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[22] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[23] J. G. A. Barbedo, “Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification,” Computers and Electronics in Agriculture, vol. 153, pp. 46–53, 2018, doi: 10.1016/j.compag.2018.08.013.

[24] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987, doi: 10.1016/0377-0427(87)90125-7.

[25] D. L. Davies and D. W. Bouldin, “A cluster separation measure,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-1, no. 2, pp. 224–227, 1979, doi: 10.1109/TPAMI.1979.4766909.

[26] T. Caliński and J. Harabasz, “A dendrite method for cluster analysis,” Communications in Statistics, vol. 3, no. 1, pp. 1–27, 1974, doi: 10.1080/03610927408827101.

[27] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in ML-based science,” arXiv:2207.07048, 2022. [Online]. Available: https://arxiv.org/abs/2207.07048.

[28] scikit-learn developers, “Model persistence,” official documentation. [Online]. Available: https://scikit-learn.org/stable/model_persistence.html. Accessed: Oct. 11, 2026.

[29] scikit-learn developers, “Common pitfalls and recommended practices,” official documentation. [Online]. Available: https://scikit-learn.org/stable/common_pitfalls.html. Accessed: Oct. 11, 2026.

[30] scikit-learn developers, “TimeSeriesSplit,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html. Accessed: Oct. 11, 2026.

[31] scikit-learn developers, “GroupShuffleSplit,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupShuffleSplit.html. Accessed: Oct. 11, 2026.

[32] Pallets Projects, “Flask documentation,” official documentation. [Online]. Available: https://flask.palletsprojects.com/en/stable/. Accessed: Oct. 11, 2026.

[33] MongoDB, “PyMongo driver documentation,” official documentation. [Online]. Available: https://www.mongodb.com/docs/languages/python/pymongo-driver/current/. Accessed: Oct. 11, 2026.

[34] scikit-learn developers, “RandomForestClassifier,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html. Accessed: Oct. 11, 2026.

[35] scikit-learn developers, “IsolationForest,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html. Accessed: Oct. 11, 2026.

[36] pandas development team, “Working with missing data,” official documentation. [Online]. Available: https://pandas.pydata.org/docs/user_guide/missing_data.html. Accessed: Oct. 11, 2026.

[37] scikit-learn developers, “StandardScaler,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. Accessed: Oct. 11, 2026.

[38] Keras developers, “EfficientNet B0 to B7,” official documentation. [Online]. Available: https://keras.io/api/applications/efficientnet/. Accessed: Oct. 11, 2026.

[39] Keras developers, “Transfer learning and fine-tuning,” official documentation. [Online]. Available: https://keras.io/guides/transfer_learning/. Accessed: Oct. 11, 2026.

[40] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. International Conference on Learning Representations, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980.

[41] Keras developers, “EarlyStopping,” official documentation. [Online]. Available: https://keras.io/api/callbacks/early_stopping/. Accessed: Oct. 11, 2026.

[42] R. R. Selvaraju et al., “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in Proc. IEEE International Conference on Computer Vision, 2017, pp. 618–626, doi: 10.1109/ICCV.2017.74.

[43] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv:1704.04861, 2017. [Online]. Available: https://arxiv.org/abs/1704.04861.

[44] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 4510–4520, doi: 10.1109/CVPR.2018.00474.

[45] R26-IT-075 project team, “Chilli classifier category mapping,” class_names.json, undated. [Unpublished class metadata].

[46] scikit-learn developers, “KMeans,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html. Accessed: Oct. 11, 2026.

[47] scikit-learn developers, “silhouette_score,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html. Accessed: Oct. 11, 2026.

[48] scikit-learn developers, “davies_bouldin_score,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.davies_bouldin_score.html. Accessed: Oct. 11, 2026.

[49] scikit-learn developers, “calinski_harabasz_score,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.calinski_harabasz_score.html. Accessed: Oct. 11, 2026.

[50] R26-IT-075 project team, “Soil model metadata,” metadata.json, undated. [Unpublished model metadata].

[51] Keras developers, “LSTM layer,” official documentation. [Online]. Available: https://keras.io/api/layers/recurrent_layers/lstm/. Accessed: Oct. 11, 2026.

[52] NumPy developers, “numpy.hstack,” official documentation. [Online]. Available: https://numpy.org/doc/stable/reference/generated/numpy.hstack.html. Accessed: Oct. 11, 2026.

[53] XGBoost developers, “Python API reference,” official documentation. [Online]. Available: https://xgboost.readthedocs.io/en/stable/python/python_api.html. Accessed: Oct. 11, 2026.

[54] GeoPandas developers, “Projections,” official documentation. [Online]. Available: https://geopandas.org/en/stable/docs/user_guide/projections.html. Accessed: Oct. 11, 2026.

[55] pyproj developers, “Transformer,” official documentation. [Online]. Available: https://pyproj4.github.io/pyproj/stable/api/transformer.html. Accessed: Oct. 11, 2026.

[56] NASA POWER, “Application programming interface,” official documentation. [Online]. Available: https://power.larc.nasa.gov/docs/services/api/. Accessed: Oct. 11, 2026.

[57] scikit-learn developers, “classification_report,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html. Accessed: Oct. 11, 2026.

[58] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” PLOS ONE, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[59] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in Proc. 34th International Conference on Machine Learning, vol. 70, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html.

[60] scikit-learn developers, “Probability calibration,” official documentation. [Online]. Available: https://scikit-learn.org/stable/modules/calibration.html. Accessed: Oct. 11, 2026.